

Hybrid MPI+OpenMP Performance of PETSc on ARCHER2

Leyan li

Date

MSc in High Performance Computing (with Data Science)
The University of Edinburgh
2026

Abstract

This is the bit where you summarise what is in your thesis. A non-expert reader should be able to understand the area of your work, what the main challenges were and what contributions or insights you've achieved, and gauge whether this work is relevant to their interests,. Think of the abstract as a super-concentrated summary of your MSc project; advertise your work but try to avoid cliffhangers.

A good abstract fits to 1 to 2 paragraphs and takes around 200 words to get your point across. To give you an idea, this abstract is 97 words long.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aims and Research Questions	1
1.3	Contributions	1
1.4	Dissertation Structure	1
2	Background and Related Work	2
2.1	Parallel Programming Models	2
2.2	Limitations of Flat MPI	2
2.3	Hybrid MPI+OpenMP	3
2.4	ARCHER2 Node Architecture and NUMA	3
2.5	PETSc and Krylov Solvers	4
2.6	Performance Analysis Tools	5
2.7	Summary and Research Gap	5
2.8	Dissertation Report	5
3	Experimental Platform and Methodology	6
3.1	Hardware and Software Environment	6
3.2	Benchmark Problems and Drivers	6
3.3	Solver Configuration	7
3.4	Configuration Space and Selection Rules	8
3.5	Job Configuration and Thread Placement	8
4	2D and 3D Problems	9
4.1	Experimental Setup	9
4.1.1	Weak-Scaling Chains	10
4.1.2	Strong-Scaling Design	10
4.2	Weak Scaling at Constant Work per Core	10
4.2.1	2D	10
4.2.2	3D	12
4.2.3	Layout Ranking and Efficiency	13

4.3	Strong Scaling at Fixed Problem Size	15
4.3.1	2D	15
4.3.2	3D	15
4.3.3	Endpoints	17
4.4	Interpretation	18
5	Profiling and Bottleneck Analysis	20
5.1	Why a Repeated-Solve Driver	20
5.2	Analysis Methodology	20
5.3	MAP Profiling Results	21
5.4	Bottleneck Identification	23
5.4.1	What is Measured	23
5.4.2	Halo Exchange Dominates the Exposure, and Hybridisation Reduces It	23
5.4.3	Reduced share Exposure	25
5.4.4	What This Does and Does Not Establish	27
6	Discussion	28
7	Conclusion and Future Work	29
A	Supplementary Figures	30
A.1	Weak-scaling throughput	30
A.2	Fixed-size runtime grids	31
B	Stuff which is too detailed	33
C	Stuff which no-one will read	34

List of Tables

2.1	The three execution models as they apply within one ARCHER2 node. The hybrid row is the subject of this dissertation; it inherits both cost columns.	2
2.2	The four fully populated layouts against the ARCHER2 node hierarchy, with close binding. Every layout uses all 128 physical cores.	4
2.3	Operations in one GAMG-preconditioned CG step, with the PETSc event that records each and the number of calls per step measured in a representative run. A step is one preconditioner application.	5
3.1	Final iteration count of the single-node default-solver sweep on a 1000×1000 grid, for every ranks $\times$ threads ≤ 128 combination. Every row is constant: the count is fixed by the rank count alone. The 128-rank configuration reached the 10,000-iteration limit without converging. . .	7
4.1	Grids and node counts of the six scales. The fixed-size column is not limited by the filter of Section 3.4.	9
4.2	The 11 admitted size–node pairs, ordered into two weak-scaling chains.	10
4.3	Fastest full-node layout at every admitted size–node pair, by median throughput. A margin of 0.0% means flat MPI was fastest.	14
4.4	Weak-scaling efficiency (%) at the largest node count of each chain, against each layout's own one-node time.	14
4.5	Fastest full-node layout at each node count for the fixed 20M problems, with its advantage over flat MPI.	17
4.6	The 32-node endpoint of the fixed-size curves. Only time per solve compares layouts directly.	17
4.7	The two dimensions on the quantities this chapter compares them by. .	18
5.1	Per-event time and flop shares, RepeatedSolves stage, 20.25M, 16×8 .	20
5.2	Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D.	21
5.3	Divided of the Exposure share	25

5.4	Reduction	25
A.1	2D fixed-size campaign, 4500^2 and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row.	31
A.2	3D fixed-size campaign, 273^3 and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row.	32

List of Figures

2.1	Architecture of an ARCHER2 node, consisting of two 64-core AMD EPYC 7742 processors.	3
4.1	2D time per KSP iteration relative to the 128×1 baseline. Values below 1 mean the hybrid layout is faster.	11
4.2	Median 2D time per KSP iteration along the two unknowns/core chains.	11
4.3	3D time per KSP iteration relative to the 128×1 baseline. Unlike 2D, no series is monotone.	12
4.4	Median 3D time per KSP iteration along the two unknowns/core chains.	13
4.5	2D time per solve at a fixed 20.25M unknowns. Both axes are logarithmic, so ideal strong scaling has slope -1	15
4.6	3D time per solve at a fixed 20.35M unknowns. Flat MPI and 64×2 turn and get slower; the most-threaded layouts do not.	16
4.7	3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. This ranks scaling, not speed.	16
5.1	Where the cores are during the solve, from the MAP call trees.	22
5.2	2D share of RepeatedSolves KSPSolve time spent inside halo exchange and global reduction events. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.	24
5.3	3D share of RepeatedSolves KSPSolve time spent inside halo exchange and global reduction events. Note the vertical scale: flat MPI reaches 84.6% at the largest point.	24
5.4	2D exposure removed relative to flat MPI against the resulting time per iteration.	26
5.5	3D exposure removed relative to flat MPI against the resulting time per iteration.	26
A.1	Median 2D repeated-solve throughput along the two unknowns/core chains.	30
A.2	Median 3D repeated-solve throughput along the two unknowns/core chains.	31

Acknowledgements

Here you can give credit to anyone who has enabled the progress of your MSc project, especially if this was *above and beyond* their role. Below is our acknowledgement of the people who contributed to creation of this template.

This template is a slightly modified version of the one developed by Prof. Charles Duncan for MSc students in the Dept. of Meteorology. His acknowledgement follows:

This template has been produced with help from many former students who have shown different ways of doing things. Please make suggestions for further improvements.

Chapter 1

Introduction

1.1 Motivation

1.2 Aims and Research Questions

This study measures PETSc rank–thread configurations for a Poisson problem on ARCHER2: flat MPI against hybrid MPI+OpenMP, in 2D and 3D, at four problem sizes, from 1 to 64 nodes. Rather than finding whether hybrid parallelism is the best, It asks whether hybrid execution reduces runtime, which configuration gives the lowest runtime, where the time goes, and what architectural factors explain the observed behaviour.

RQ1 Does PETSc’s threaded-kernel backend reduce the time to solution relative to flat MPI on a fully populated node, and by how much?

RQ2 If it does, at which rank–thread balances, and how does the answer change with problem size, node count, dimensionality, and scaling regime?

RQ3 Which parts of the solve account for the difference?

RQ4 How far do the observed trends generalise, and what would be needed to establish a mechanism for them?

1.3 Contributions

1.4 Dissertation Structure

Chapter 2

Background and Related Work

2.1 Parallel Programming Models

A hybrid program assigns one or more MPI ranks to each node and creates an OpenMP team inside every rank.

Table 2.1: The three execution models as they apply within one ARCHER2 node. The hybrid row is the subject of this dissertation; it inherits both cost columns.

Model	Memory	Data exchange	Principal cost
MPI	Private per rank	Explicit messages	Message matching, per-rank solver state, one subdomain boundary per rank
OpenMP	Shared within a team	Shared addresses	Team creation, synchronisation, cache and bandwidth contention
Hybrid	Shared inside a rank, private between ranks	Messages between ranks only	Both, traded against each other by the rank–thread balance

2.2 Limitations of Flat MPI

Flat MPI assigns one rank to each core.

2.3 Hybrid MPI+OpenMP

PETSc –with-openmp-kernels

2.4 ARCHER2 Node Architecture and NUMA

An ARCHER2 compute node has two AMD (Rome) processors, giving 128 physical cores. The service configures four NUMA regions per socket, eight per node, each with 16 cores and local memory channels in each region. A core has private L1 and L2 caches. Four cores forms a Core Complex (CCX) and share a 16MB L3 cache, two CCXs form a Core Complex Die (CCD). Different CCXs do not share L3 cache [1].

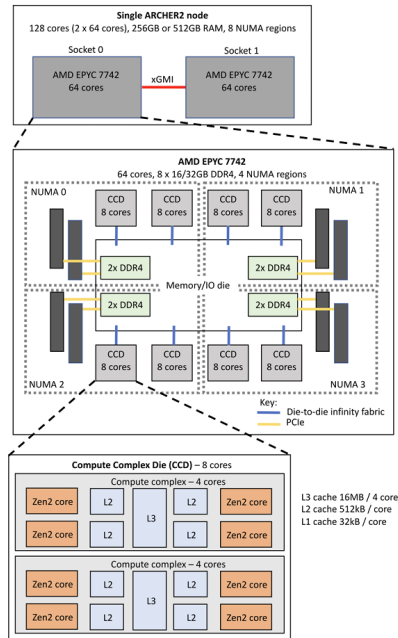


Figure 2.1: Architecture of an ARCHER2 node, consisting of two 64-core AMD EPYC 7742 processors.

NUMA affects each both latency and bandwidth. Linux normally allocates a page in the NUMA region of the thread that first touches it. If a thread creates a large array and a team accesses it later later from several regions, remote traffic can concentrate on the memory controllers of the first region. The ARCHER2 guide also notes that two threads can approach the memort-bandwidth limit of a NUMA region for bandwidth-intensive code [2]. More threads do not guarantee more bandwidth.

Cache topology introduces a finer boundary than NUMA. Four adjacent cores share an L3 cache, whereas a 16-core NUMA region countains four such L3 caches. With

Table 2.2: The four fully populated layouts against the ARCHER2 node hierarchy, with close binding. Every layout uses all 128 physical cores.

Layout	Cores/rank	Subdomains/node	Rank against the shared-L3 (CCX) boundary
128×1	1	128	four ranks share one CCX
64×2	2	64	two ranks share one CCX
32×4	4	32	exactly one CCX
16×8	8	16	spans two CCXs

close binding, 32×4 can place one four-thread rank within on CCX; 64×2 can use two two-thread rank to share one CCX. 16×8 can make a rank span two CCXs, and 128×1 can make four ranks share one CCX. This mapping is a plausible mechanism for layout differences, The experiment fix thread affinity and compare measuremetns before using topology to interpret the results, The relevant binding options are:

```
--cpu-bind=cores
--distribution=block
--hint=nomultithread
```

2.5 PETSc and Krylov Solvers

PETSc provides distributed vectors and matrices, Krylov solvers (KSP), preconditioners (PC), and a built-in logging system. It talks between processes with MPI, and it lets the user pick solver components at run time.

The production system is the finite-difference Laplacian, whose matrix is a sparse, symmetric, and positive definite. Conjugate gradients (CG) is designed for this type of linear system. At each iteration CG applies the matrix and the preconditioner, updates vectors, and evaluates inner products. Table 2.3 classifies the work of one CG step accordingly. The counts show why one step is dominated by neighbour exchange rather than by the two CG dot products: the multigrid hierarchy performs its own halo exchange on every level, so the 42.9 exchanges per step is more lager than the 3 golbal reductions. Neighbour and global costs scale differently with the rank count, The Section 5.4 separates them and measures them both.

A preconditioner reshapes the spectrum of the system, so CG reaches the requested tolerance in fewer iterations. Algebraic multigrid (AMG) builds a hierarchy of coarser problems directly from the matrix. On each level a smoother quickly removes high-frequency error; the smooth error that a local smoother handles slowly is instead restricted to a coarser level, corrected there, and prolonged back. PETSc's GAMG (geometric-algebraic multigrid, PCGAMG) builds this hierarchy without needing a geometric description of the mesh.

Table 2.3: Operations in one GAMG-preconditioned CG step, with the PETSc event that records each and the number of calls per step measured in a representative run. A step is one preconditioner application.

Operation	PETSc event	Communication	Calls/step
Matrix-vector product	MatMult	neighbour	28.9
Multigrid restriction	MatMultTranspose	neighbour	7.0
Multigrid prolongation	MatMultAdd	neighbour	7.0
Level residual	MatResidual	neighbour	7.0
Halo exchange, all of the above	VecScatterBegin	neighbour	42.9
Inner products	VecTDot	global	2.0
Residual norm	VecNorm	global	1.0
Vector updates	VecAXPY, VecAYPX	none	44.6

2.6 Performance Analysis Tools

`log_view` and Linaro MAP are the two PETSc tools used in this study. `log_view` is a built-in logging system that can report the time spent in each PETSc function, while Linaro MAP is a sampling profiler for MPI, OpenMP, and single-threaded programs that provides detailed information about the performance of PETSc applications, including memory usage, communication patterns, and computational efficiency.

2.7 Summary and Research Gap

2.8 Dissertation Report

Chapter 3

Experimental Platform and Methodology

3.1 Hardware and Software Environment

All experiments ran on ARCHER2. Each node has two AMD EPYC 7742 processors, 128 physical cores, and eight NUMA regions of 16 cores each. The software stack was PrgEnv-gnu 8.4.0, gcc 11.2.0, and cray-mpich 8.1.27.

PETSc 3.24.4 was built from source `asarch-omp-opt`. The repeated-solve campaigns used `asarch-omp-libsci-rome-opt`, configured with the threaded `libsci_gnu_mp.so.5.0`.

LibSci is HPE Cray's tuned implementation of the Basic Linear Algebra Subprograms (BLAS) and LAPACK, the standard interfaces for dense vector-vector, matrix-vector and matrix-matrix operations; the `_mp` variant threads those kernels with OpenMP.

Both with PETSC's OPENMP kernels and use `-O3`. One thread per rank is used as the MPI-only baseline.

3.2 Benchmark Problems and Drivers

The 2D benchmark is PETSc `ex2`, a five-point stencil discretisation of the Laplace operator on an $m \times n$ grid. The 3D benchmark is a driver written for this project, a seven-point stencil on an $m \times n \times p$ grid; it was added because surface-to-volume ratio differs from 2D. Its production driver, `ex2_repeat`, separates one warm-up solve from 19 timed solves with a reused preconditioner (Chapter 5). The matched `3d_repeat` driver applies the same measurement design to a seven-point stencil on an $m \times n \times p$ grid.

3.3 Solver Configuration

The default ex2 solver is GMRES with block `Jacobi.Block`. Jacobi builds one block per MPI rank, so the preconditioner is a function of the rank decomposition: changing the rank–thread layout changes the operator that is applied. The dependence is not monotonic in the rank count, so it cannot be read as a trade-off between preconditioner strength and parallelism either. Two layouts with different rank counts apply different preconditioners to the same linear system, and a wall-clock difference between them reflects that difference as much as any property of the layout. Cross-layout runtime under this solver is therefore indicative only, and the question “which rank–thread layout is fastest?” has no well-posed answer. Table 3.1 shows this directly. Reading across any row, the iteration count does not change at all when threads are added; reading down any column, it changes by a factor of more than three.

Table 3.1: Final iteration count of the single-node default-solver sweep on a 1000×1000 grid, for every ranks $\times$ threads ≤ 128 combination. Every row is constant: the count is fixed by the rank count alone. The 128-rank configuration reached the 10,000-iteration limit without converging.

MPI ranks	OpenMP threads per rank							
	1	2	4	8	16	32	64	128
1	6700	6700	6700	6700	6700	6700	6700	6700
2	3715	3715	3715	3715	3715	3715	3715	
4	2825	2825	2825	2825	2825	2825		
8	4085	4085	4085	4085	4085			
16	8179	8179	8179	8179				
32	6460	6460	6460					
64	7181	7181						
128	10000							

CG is used because it is a common Krylov method for symmetric positive definite problems, and GAMG removes that coupling.

```
-ksp_type cg -pc_type gamg -ksp_rtol 1e-5 -ksp_atol 1e-10
```

Its iteration count is set by the multigrid hierarchy and not by the rank decomposition.

GAMG cost has two phases: set up (`PCSetUp`) and solve (`KSPSolve`). These two phases carry different costs, so the production campaign times repeated solves with the hierarchy reused rather than first solves; Section 5.1 gives the measured split and the driver that produces it.

3.4 Configuration Space and Selection Rules

Three rules are followed in this experiment:

1. Threads per rank are restricted to $\{1, 2, 4, 8\}$.
2. Nodes are fully populated: $\text{ppn} \times \text{threads} = 128$, giving four layouts 128×1 , 64×2 , 32×4 , and 16×8 . Same node count means same core count, so a comparison isolates rank–thread balance from core count.
3. A configuration at size N on C physical cores is included when $25,000 \leq N/C \leq 100,000$. The completed 2D campaign uses six grids: 2240^2 , 3160^2 , 4500^2 , 6400^2 , 9073^2 , and 12828^2 , or 5.02M to 164.56M unknowns.

3.5 Job Configuration and Thread Placement

Each rank and its threads are pinned to a contiguous set of cores with:

```
OMP_PLACES=cores    OMP_PROC_BIND=close  
srun --hint=nomultithread --distribution=block:block --exact
```

The primary timing is the maximum-rank KSPSolve time in the RepeatedSolves stage. If this event has count $c = 19$, time t , and the grid has N unknowns, steady-state throughput is

$$Q = \frac{Nc}{t}.$$

The analysis also reports $t/(cI)$, where I is the final KSP iteration count. Formal repeats are aggregated by their median; **whiskers** show the observed minimum and maximum from three runs. Scaling is computed per layout against that layout’s smallest admitted node count. For a layout ℓ at n nodes,

$$S_\ell(n) = \frac{T_\ell(n_0)}{T_\ell(n)}, \quad E_\ell(n) = \frac{S_\ell(n)}{n/n_0},$$

where n_0 is the smallest node count run for that problem size.

Chapter 4

2D and 3D Problems

4.1 Experimental Setup

Two campaigns cover each dimension. Both use the platform, build, solver and thread placement of Chapter 3, and both fill every node with all 128 cores. They differ in one thing. The weak-scaling campaign grows the problem with the node count. The fixed-size campaign holds it still. Table 4.1 lists both.

Table 4.1: Grids and node counts of the six scales. The fixed-size column is not limited by the filter of Section 3.4.

Scale	2D grid	2D unknowns	3D grid	3D unknowns	Weak-scaling nodes	Fixed-size nodes
5m	2240^2	5,017,600	171^3	5,000,211	1	—
10m	3160^2	9,985,600	215^3	9,938,375	1, 2	—
20m	4500^2	20,250,000	273^3	20,346,417	2, 4	1, 2, 4, 8, 16, 32
41m	6400^2	40,960,000	345^3	41,063,625	4, 8	—
82m	9073^2	82,319,329	435^3	82,312,875	8, 16	—
165m	12828^2	164,557,584	548^3	164,566,592	16, 32	—

That gives 11 size–node pairs in the weak-scaling campaign and six in the fixed-size one. Every pair runs all four layouts three times. All 280 weak-scaling logs and all 144 fixed-size logs passed the content checks. None carries a divergence, a PETSc or MPI error, or a licence wait.

Run-to-run spread limits what can be resolved. Over three repeats the largest spread is 4.71% in 2D and 3.83% in 3D for the weak-scaling matrix. For the fixed-size curves it is 10.77% and 11.09%, because a solve at the largest node counts takes only tens of milliseconds. A gap smaller than these is reported as a tie. Whiskers in the figures show the observed range of the three runs.

4.1.1 Weak-Scaling Chains

The filter $25,000 \leq N/C \leq 100,000$ admits each scale at only one or two node counts, so a panel drawn per problem size holds two points. The same 11 pairs split into two chains of near-constant unknowns per core (Table 4.2). Ideal weak scaling would hold the time per iteration flat along a chain. The measured rise is the cost of concurrency.

Table 4.2: The 11 admitted size–node pairs, ordered into two weak-scaling chains.

Band	Size at node count					
	1	2	4	8	16	32
~40k unknowns/core	5m	10m	20m	41m	82m	165m
~80k unknowns/core	10m	20m	41m	82m	165m	—

4.1.2 Strong-Scaling Design

The chains span one node doubling per problem size. That cannot show where a layout stops scaling. A second campaign therefore fixes the 20m problem and runs the full sequence $\{1, 2, 4, 8, 16, 32\}$ nodes, with nothing else changed. At 32 nodes this leaves about 4.9 thousand unknowns per core, below the admission window. That point is the saturation tail and not a sensible operating point.

4.2 Weak Scaling at Constant Work per Core

Both dimensions run the same 11 pairs, the same four layouts and the same three repeats.

4.2.1 2D

Throughput rises almost linearly with the node count (Figure A.1). All four layouts sit within a few per cent of each other on that axis, so it cannot separate them. Figure 4.1 divides each layout’s time per iteration by that of flat MPI at the same size–node pair. This removes the scale and the GAMG iteration count and leaves the layout effect.

The curves fall. Along the ~40k chain 64×2 goes from 0.957 at one node to 0.872 at 16, and 32×4 from 1.096 to 0.937. Along the ~80k chain all three hybrid ratios drop at every step. Four of the six series are strictly monotone.

Two things follow. 64×2 beats flat MPI per iteration at all 11 pairs, and its lead grows with the node count. 32×4 starts 9.6% behind flat MPI, reaches parity at eight

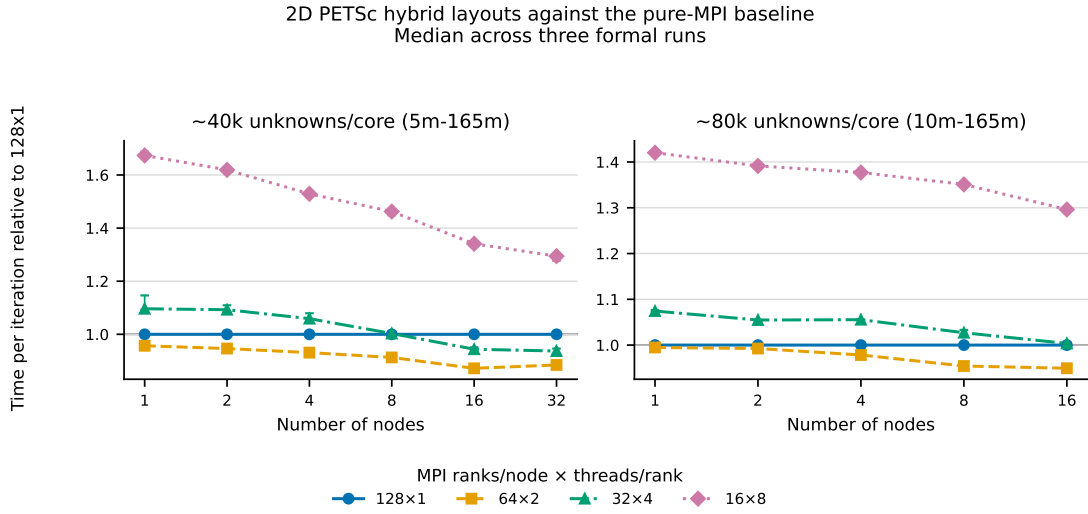


Figure 4.1: 2D time per KSP iteration relative to the 128×1 baseline. Values below 1 mean the hybrid layout is faster.

nodes, and is 6.3% ahead at 32 nodes. A ranking taken at one node does not hold at 32.

16×8 is the slowest layout at every pair. Its deficit does narrow, from 46.2% at 5m on one node to 22.7% at 165m on 32 nodes.

Figure 4.2 gives the absolute time per iteration. A flat line would be ideal weak scaling.

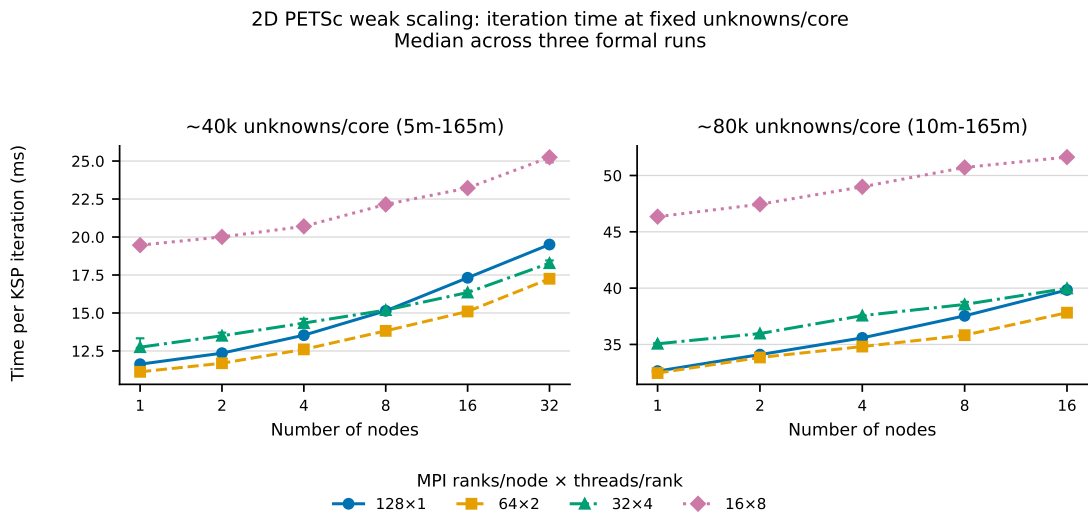


Figure 4.2: Median 2D time per KSP iteration along the two unknowns/core chains.

4.2.2 3D

The 3D campaign uses the same points, layouts and repeats. Its throughput has the same near-linear shape (Figure A.2), so the comparison again uses the ratio (Figure 4.3).

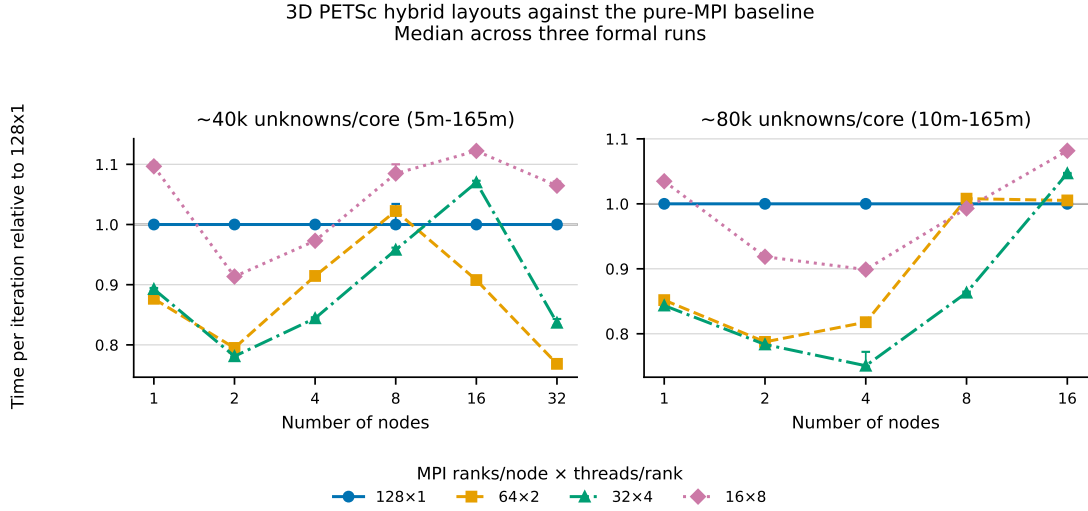


Figure 4.3: 3D time per KSP iteration relative to the 128×1 baseline. Unlike 2D, no series is monotone.

3D differs from 2D in size and in shape. The best point is 32×4 at 41m on four nodes, at a ratio of 0.751. That is 24.9% less time per iteration than flat MPI, against a 2D best of 12.8%. Of the 33 hybrid measurements 22 fall below 1, against 13 of 33 in 2D.

The shape differs too. None of the six series is monotone. On the $\sim 80k$ chain the hybrid layouts lead at two and four nodes, lose the lead by eight nodes, and are at or above parity at 16. On the $\sim 40k$ chain 64×2 rises to 1.022 at 41m on eight nodes and then falls back to 0.768 at 165m on 32.

16×8 wins no point. Unlike the 2D case it is not always last, and it beats flat MPI at five of the 11 pairs.

Figure 4.4 gives the absolute time per iteration, on the same convention as the 2D case.

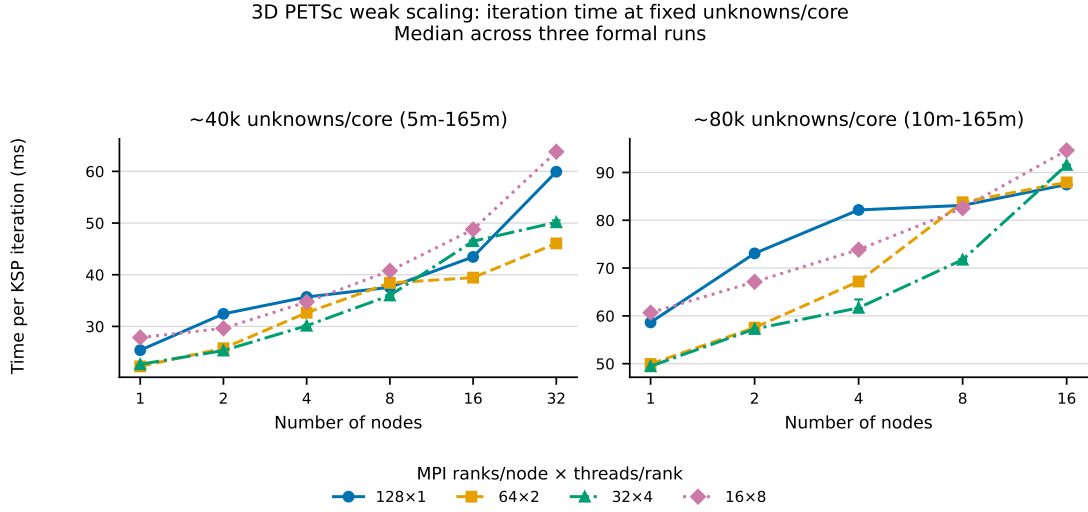


Figure 4.4: Median 3D time per KSP iteration along the two unknowns/core chains.

4.2.3 Layout Ranking and Efficiency

Table 4.3 gives the fastest layout at every pair and its margin over flat MPI.

64×2 wins 8 of the 11 2D pairs. In 3D 32×4 wins 6, 64×2 wins 4 and flat MPI wins 1. The 2D winner changes twice down the column and the 3D winner changes five times. 2D has one layout ahead throughout. 3D does not. One of the three 2D flat-MPI wins, 41m on eight nodes, is a tie: the medians differ by 0.4% and the ranges overlap.

Table 4.4 measures each layout against its own one-node time.

In 2D the ordering is strict in both chains. Efficiency rises with threads per rank, and 16×8 degrades least of the four. In 3D there is no such ordering. On the $\sim 40k$ chain 64×2 degrades least, and on the $\sim 80k$ chain flat MPI does.

16×8 has the best 2D efficiency and the worst 2D time at every point. It degrades least because its cost is already dominated by a term that does not grow with the node count. Efficiency measured against a layout's own baseline rewards a bad baseline.

Table 4.3: Fastest full-node layout at every admitted size–node pair, by median throughput. A margin of 0.0% means flat MPI was fastest.

Scale	Nodes	Cores	Best layout	M eq/s	vs. MPI	ms/iteration
<i>2D, five-point stencil</i>						
5m	1	128	128×1	47.9	0.0%	11.63
10m	1	128	128×1	34.0	0.0%	32.63
10m	2	256	64×2	94.9	5.7%	11.69
20m	2	256	64×2	59.8	0.8%	33.84
20m	4	512	64×2	160.7	7.4%	12.60
41m	4	512	64×2	107.0	2.2%	34.81
41m	8	1024	128×1	270.5	0.0%	15.14
82m	8	1024	64×2	208.9	4.8%	35.82
82m	16	2048	64×2	495.6	14.7%	15.10
165m	16	2048	64×2	362.6	5.3%	37.82
165m	32	4096	64×2	794.8	13.1%	17.25
<i>3D, seven-point stencil</i>						
5m	1	128	64×2	37.4	14.1%	22.26
10m	1	128	32×4	28.7	1.6%	49.45
10m	2	256	64×2	64.3	25.8%	25.78
20m	2	256	32×4	50.8	27.6%	57.26
20m	4	512	32×4	96.5	1.5%	30.13
41m	4	512	32×4	95.1	33.2%	61.68
41m	8	1024	32×4	163.0	4.4%	35.99
82m	8	1024	32×4	163.9	15.8%	71.75
82m	16	2048	64×2	298.2	10.2%	39.44
165m	16	2048	128×1	268.8	0.0%	87.47
165m	32	4096	64×2	510.4	30.1%	46.06

Table 4.4: Weak-scaling efficiency (%) at the largest node count of each chain, against each layout’s own one-node time.

Case	Chain and node count	Layout			
		128×1	64×2	32×4	16×8
2D	~40k, 32 nodes	59.6	64.5	69.8	77.1
2D	~80k, 16 nodes	81.9	85.8	87.7	89.8
3D	~40k, 32 nodes	42.4	48.3	45.2	43.7
3D	~80k, 16 nodes	67.0	56.8	54.0	64.1

4.3 Strong Scaling at Fixed Problem Size

4.3.1 2D

The 2D curves scale well and stay together (Figure 4.5). Every layout is still improving at 32 nodes. The spread between fastest and slowest goes from $1.47\times$ at one node to $1.81\times$ at 16 and back to $1.48\times$ at 32. 64×2 and flat MPI finish within 2% of each other. 16×8 starts 47% behind and ends 45% behind.

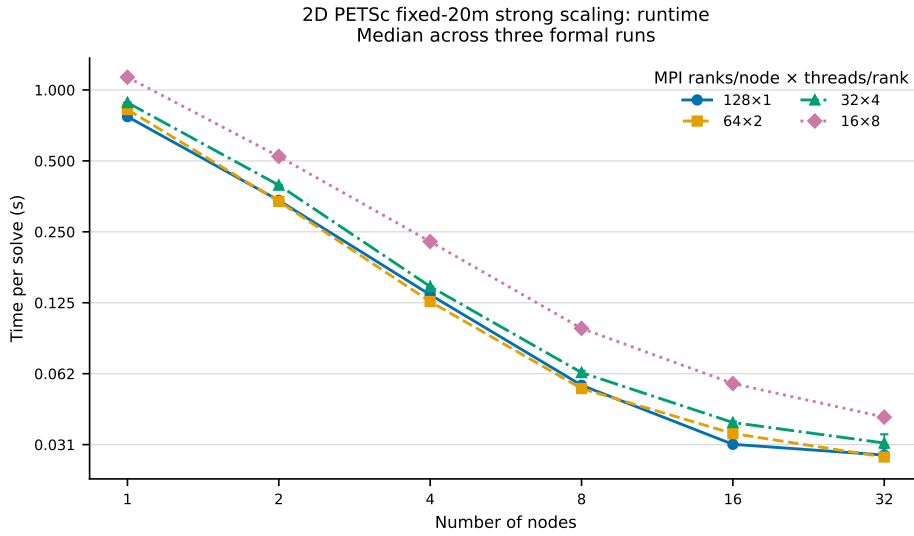


Figure 4.5: 2D time per solve at a fixed 20.25M unknowns. Both axes are logarithmic, so ideal strong scaling has slope -1 .

Cumulative efficiency passes 100% between two and 16 nodes and peaks at 191% for 64×2 at eight nodes. These superlinear values repeat across the three runs. This experiment does not explain them. The fixed problem shrinking into cache is one candidate it cannot test.

4.3.2 3D

The 3D result is different in kind, and it is the largest layout effect in this study (Figure 4.6). Every 3D layout eventually turns and gets slower. They turn at different node counts, and the order is the thread count.

Flat MPI turns first. It bottoms at eight nodes with 0.0997s per solve, then rises to 0.1776s at 16 nodes and 0.2599s at 32. 64×2 and 32×4 bottom at 16 nodes. 16×8 has not turned by 32 nodes. There it is the fastest of the four at 0.0681s per

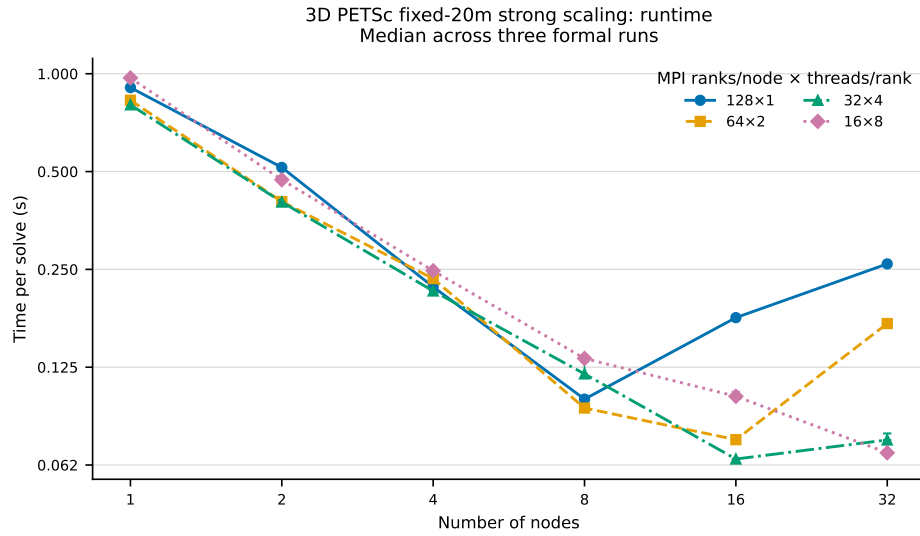


Figure 4.6: 3D time per solve at a fixed 20.35M unknowns. Flat MPI and 64×2 turn and get slower; the most-threaded layouts do not.

solve, $3.81\times$ faster than flat MPI on the same 4096 cores. The observed ranges at that point do not overlap.

The saturation node count rises with threads per rank: 8, 16, 16, and beyond 32. That is what hybridisation buys in 3D.

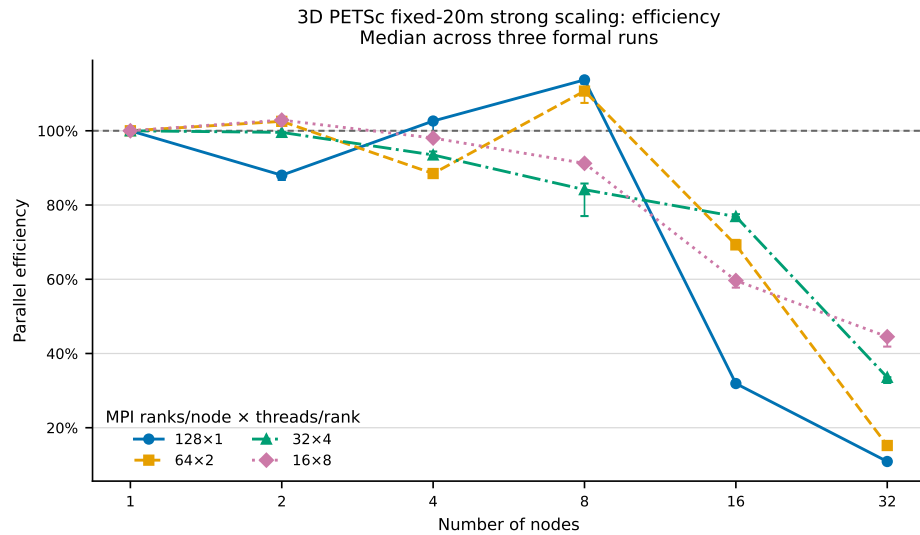


Figure 4.7: 3D parallel efficiency at a fixed 20.35M unknowns, each layout against its own one-node time. This ranks scaling, not speed.

Figure 4.7 shows the same runs as parallel efficiency. Each layout is measured against its own one-node time, so it ranks scaling and not speed. No 2D counterpart is drawn, because the 2D curves do not turn over.

4.3.3 Endpoints

Table 4.5 gives the fastest layout at each node count. In 2D the margin over flat MPI never exceeds 6.9%. In 3D it reaches 172.4% at 16 nodes and 281.3% at 32.

Table 4.5: Fastest full-node layout at each node count for the fixed 20M problems, with its advantage over flat MPI.

Nodes	Cores	2D, 4500 ²			3D, 273 ³		
		Best	s/solve	vs. MPI	Best	s/solve	vs. MPI
1	128	128 × 1	0.7704	0.0%	32 × 4	0.8022	13.0%
2	256	64 × 2	0.3365	1.0%	32 × 4	0.4028	27.8%
4	512	64 × 2	0.1265	6.9%	32 × 4	0.2145	2.9%
8	1024	64 × 2	0.0541	3.3%	64 × 2	0.0936	6.5%
16	2048	128 × 1	0.0313	0.0%	32 × 4	0.0652	172.4%
32	4096	64 × 2	0.0277	1.9%	16 × 8	0.0681	281.3%

Table 4.6 gives all four layouts at 32 nodes. The 2D speedups are almost equal, 27.3 to 29.8, because each layout is measured against its own one-node time and 16 × 8 starts from the worst one. Their runtimes still differ by 45%. In 3D the two measures agree, because the layouts diverge instead of converging.

Table 4.6: The 32-node endpoint of the fixed-size curves. Only time per solve compares layouts directly.

Case	Layout	s/solve	Speedup	Efficiency	16→32 step
2D	128 × 1	0.0282	27.30	85.3%	55.5%
2D	64 × 2	0.0277	29.83	93.2%	62.9%
2D	32 × 4	0.0317	27.85	87.0%	61.1%
2D	16 × 8	0.0409	27.71	86.6%	69.4%
3D	128 × 1	0.2599	3.49	10.9%	34.2%
3D	64 × 2	0.1703	4.87	15.2%	22.0%
3D	32 × 4	0.0747	10.75	33.6%	43.7%
3D	16 × 8	0.0681	14.24	44.5%	74.6%

4.4 Interpretation

Table 4.7 collects the comparison. It gives both a throughput gain and a time saving, because these are different conventions and the chapter quotes both. The two halves do not carry equal weight. The 3D gaps over flat MPI at 32 nodes, $3.81\times$, are far outside the run-to-run spread. The 1–7% margins the 2D layouts hold on the fixed problem are not, so the 2D ordering at any single node count is only indicative.

Table 4.7: The two dimensions on the quantities this chapter compares them by.

	2D	3D
<i>Constant work per core, 11 size–node pairs</i>		
Layout winning most points	64×2 (8 of 11)	32×4 (6 of 11)
Hybrid points faster per iteration	13 of 33	22 of 33
Largest throughput gain	14.7%	33.2%
Largest per-iteration saving	12.8%	24.9%
Ratio series monotone in nodes	4 of 6	0 of 6
Weak efficiency rises with threads	yes	no
<i>Fixed 20M problem, one to 32 nodes</i>		
Layouts whose time turns rise	none of 4	3 of 4
First to rise	—	128×1 , at 8 nodes
Fastest at 32 nodes	64×2	16×8
Its lead over flat MPI	$1.02\times$	$3.81\times$
Fastest-to-slowest spread at 32 nodes	$1.48\times$	$3.81\times$

2D is ordered and 3D is not. Every 2D hybrid layout improves against flat MPI as concurrency grows, and weak efficiency rises with threads per rank. In 3D the margins are about twice as large, but no series is monotone and the best layout changes with scale and node count.

The regime matters as much as the dimension. At constant work per core the layout effect is a few per cent. At constant problem size it is unbounded, because the layouts differ in where they stop scaling rather than in speed.

The two stencils do different work per equation, so their absolute rates are not a comparison of algorithms. What compares is the layout ordering and the margin over flat MPI.

Two results here are not explained here. The first is the 2D ordering. Adding threads helps more as concurrency grows, which is what a communication-share account would predict. This chapter measures runtime and not communication, so it cannot test that account. The second is 16×8 . It removes the most MPI endpoints of any layout and is still the slowest 2D layout at all 11 points. Whatever hybridisation saves, it also costs something that grows with the thread count. Chapter 5 measures both directly.

What this chapter does settle is that a layout recommendation needs the regime it was measured in. 16×8 is last at constant work per core in both dimensions and first at 4096 cores on the fixed 3D problem, by $3.81\times$. Both readings are right. They answer different questions.

Chapter 5

Profiling and Bottleneck Analysis

5.1 Why a Repeated-Solve Driver

In a single solve at 20.25M with 16×8 , `-log_view` gave PCSetUp 45% of time and KSPSolve 10%. `ex2_repeat` runs the solve inside a RepeatedSolves log stage, zeroes the initial guess before each solve, and reuses the preconditioner (`-ksp_reuse_preconditioner` and `-pc_gamg_reuse_interpolation true`). With 20 solves the stage held 69% of time and 93% of flops; within the stage KSPSolve was 100% of time; each of the 19 repeats took 11 iterations. The corrected scaling campaign uses this same warm-up and reuse design, so Chapter 4 compares steady-state solves rather than first-touch and hierarchy-construction costs.

5.2 Analysis Methodology

The method combines MAP 25.0.4 timelines with `-log_view %T` (share of stage time) and `%F` (share of stage flops). An event whose `%T` exceeds its `%F` spends the difference on communication, synchronisation, or memory traffic. Table 5.1 lists single-point data for the RepeatedSolves stage at 20.25M with 16×8 .

Table 5.1: Per-event time and flop shares, RepeatedSolves stage, 20.25M, 16×8 .

Event	%T	%F	GFlop/s
MatMult	39	55	30.5
MatMultTranspose	18	6	6.7
VecTDot+VecNorm	8	6	–
PCApply (agg.)	82	82	21.6

MatMultTranspose (GAMG restriction) has a `%T` three times its `%F` and a rate

4.6× below MatMult. VecTDot and VecNorm are the CG global reductions.

5.3 MAP Profiling Results

Two MAP campaigns profiled the same 20.25M 2D problem:

1. a baseline set on arch-omp-opt at 2, 8, 16 nodes
2. a matching set on the production arch-omp-libsci-rome-opt at 2 and 8 nodes.

Neither contains a 128×1 is because MAP cannot launch 128 MPI ranks per node on this system. So the Flat MPI has `-log_view` evidence only. The solve window scope keeps only samples in which cores are inside KSPSolver and ignore PCSetUp, which approximates the repeated-solve phase from the profile alone. This is a time window, not the PETSc RepeatedSolves logging stage, and the two must not be recognized as the same quantity.

Table 5.2: Share of total core-time inside the solve window, by the library owning the executing frame, for the eleven usable MAP profiles at 20.25M unknowns in 2D.

Build	Nodes	Layout	PETSc	OpenMP	MPI	LibSci
LibSci	2	64×2	31.9%	36.5%	29.9%	1.0%
LibSci	2	32×4	51.6%	40.1%	5.8%	1.7%
LibSci	2	16×8	42.2%	53.5%	1.4%	2.1%
LibSci	8	64×2	4.5%	48.3%	46.7%	0.2%
LibSci	8	32×4	7.5%	69.3%	22.6%	0.3%
LibSci	8	16×8	15.6%	76.5%	6.2%	1.0%
Baseline	2	32×4	51.7%	40.8%	6.7%	0.0%
Baseline	2	16×8	40.2%	57.8%	1.4%	0.0%
Baseline	8	32×4	8.8%	68.6%	22.2%	0.0%
Baseline	8	16×8	16.0%	77.5%	6.0%	0.0%

The two costs trade against each other. The MPI share falls as threads replace ranks: at eight nodes it goes 46.7%, 22.6%, 6.2% across 64×2 , 32×4 and 16×8 . That is the ordering Section 5.4 obtains from `-log_view` on the whole formal matrix, reproduced here by a different instrument on a different quantity. The time spent in the OpenMP runtime runs the other way, 48.3%, 69.3%, 76.5%, and it is not useful work: the frames are `cpu_relax`, `do_spin` and `__futex_wait`, that is threads waiting for something to do. The two builds agree wherever they overlap: across the four common cells the MPI shares differ by at most 0.9 percentage points and the OpenMP shares by at most 4.3, with no systematic sign.

Also MAP's limit also should be considered, There is one profile per pair, and the overhead of MAP is large and uneven (separately measured at +35% on the timed stage and roughly threefold on VecScatterEnd), and its OpenMP instrumentation influence the barrier behaviour being measured, so the absolute percentages overstate what an uninstrument run would show.

Consider those limits the table answer the question Chapter 4, at the level of accounting rather than of mechanism. The cost that grows with threads per rank, and that must be repaid before hybridisation worked, is paid as the idle time in the openMP runtime: the term that grows is time in `cpu_relax`, `do_spin` and `__futex_wait`, not time in the compute kernels. NUMA placement is excluded on topology alone: a rank of eight threads crosses a shared-L3 boundary but still fits in side one 16-cores NUMA region, so no layout of this study places a rank across two NUMA regions and the four cannot differ in that respect (Table 2.2).

Neither is the growth a placement effect, because it is present at fixed layout, where a rank's threads occupy the same cores against the same shared-L3 boundary at every node count: holding 16×8 and going from two nodes to eight raises the waiting share from 53.5% to 76.5% on the LibSci build and from 57.8% to 77.5% on the baseline, as the work per threaded region shrinks.

The other two features of the shape, monotonicity in the thread count and a maximum exactly where 16×8 performs worst, are consistent with this reading but equally consistent with a cache one. What the profiles do not settle is why the threads wait: a rank spanning two CCXs could produce imbalance that is collected at the same barrier, and Section 5.4.4 leaves that to the hardware-counter work of Chapter 7. Section 5.4 shows the other half of the trade, on the full matrix and without a profiler.

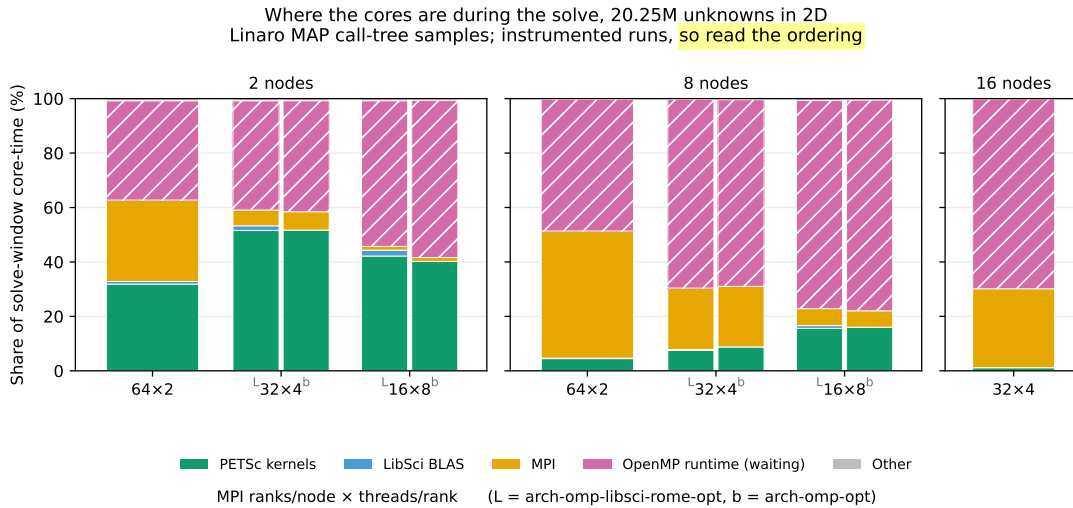


Figure 5.1: Where the cores are during the solve, from the MAP call trees.

The last observation is negative and worth considering. Threaded Libsci is linked into the production build, and PETSC's sparse solve path barely calls it: BLAS accounts for at most 2.1% of solve-window core-time in any Libsci profile, and the baseline profiles sample no BLAS frame at all. The threaded BLAS library therefore cannot account for the layout ordering measured in Chapter 4, which confirms from profile data what Section 5.4.4 could previously only state as a limit of the evidence.

5.4 Bottleneck Identification

Section 5.3 leaves the hardware question open, but the `-log_view` event tables of the uninstrumented formal runs support a question that Chapter 4 raised: whether the layout effect tracks the share of the solve exposed to inter-process synchronisation. All formal runs are parsed here, so the evidence covers the same matrix as the scaling results and carries no profiler overhead.

5.4.1 What is Measured

For each run, the time of the halo-exchange events (`VecScatterBegin`, `VecScatterEnd`) and of CG's global-reduction events (`VecTDot`, `VecNorm`) is taken from the `RepeatedSolves` stage and divided by that stage's `KSPSolve` time.

Also some things of the `-log_view` should be considered:

The synchronising events are logged inside `MatMult` and its relatives, so this is an exposure share. These are not a term in an additive time budget. Summing every leaf event with `KSPSolve` gives a result that larger than 100%.

Each event time is a maximum over ranks, and different events may attain their maximum on different ranks. `VecTDot` and `VecNorm` also perform local flops, so their time is not purely communication.

The quantity is measured identically in every run, which is what makes it comparable across layouts and node counts.

5.4.2 Halo Exchange Dominates the Exposure, and Hybridisation Reduces It

Figure 5.2 and Figure 5.3 give the result. In both dimensions the share rises monotonically with node count for every layout, and these size-node points it is strictly ordered by MPI rank count: fewer ranks, less exposure. The plotted share is the sum of the two event classes, which sit on disjoint call path: the halo exchange is reached from `MatMult` and its relatives, the reductions directly from CG.

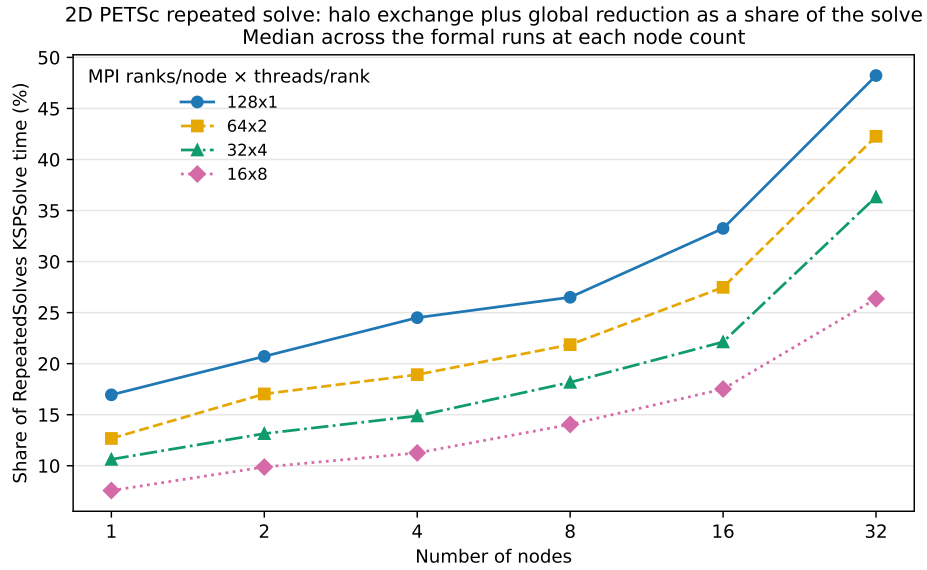


Figure 5.2: 2D share of RepeatedSolves KSPSolve time spent inside halo exchange and global reduction events. The ordering matches the time-per-iteration ordering in Figure 4.1 at every node count.

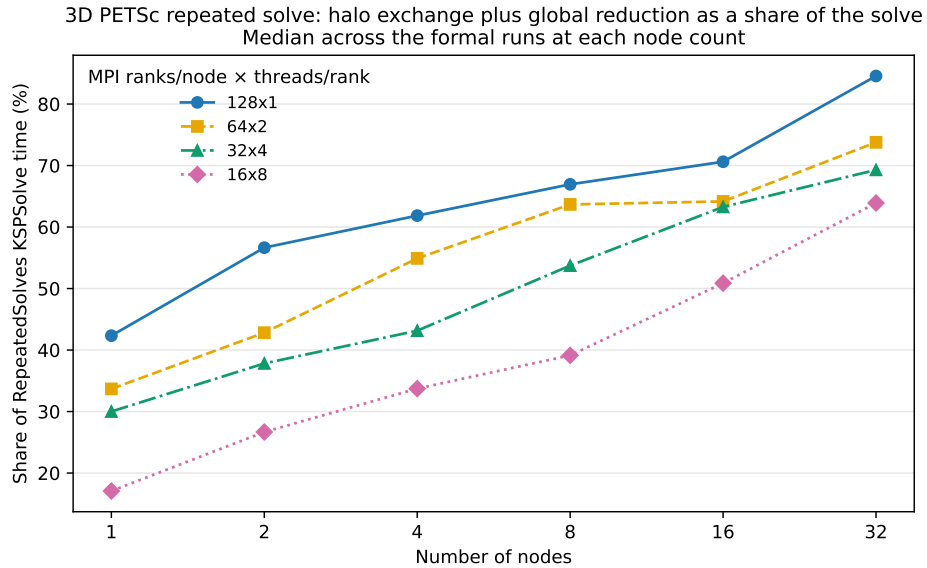


Figure 5.3: 3D share of RepeatedSolves KSPSolve time spent inside halo exchange and global reduction events. Note the vertical scale: flat MPI reaches 84.6% at the largest point.

Table 5.3: Divided of the Exposure share

Case	Nodes (size)	128×1		16×8	
		Halo	Reduction	Halo	Reduction
2D	1 (5m)	15.7	5.4	6.1	2.9
2D	8 (41m)	23.4	9.4	11.2	6.1
2D	16 (82m)	28.6	12.4	13.6	7.6
2D	32 (165m)	34.3	13.9	16.4	9.9
3D	1 (5m)	35.1	9.9	15.3	4.6
3D	8 (41m)	53.0	18.1	38.8	6.8
3D	16 (82m)	59.0	15.8	46.5	10.8
3D	32 (165m)	70.9	13.6	52.4	11.5

5.4.3 Reduced share Exposure

Table 5.4: Reduction

Case	Layout	Exposure removed (pt)	Time ratio	r	Break-even (pt)
2D	64×2	0.8–7.4	0.872–0.995	−0.67	1.2
2D	32×4	5.3–13.7	0.937–1.096	−0.59	11.7
2D	16×8	6.3–21.9	1.294–1.674	+0.00	not reached
3D	64×2	1.9–15.1	0.768–1.022	−0.86	2.9
3D	32×4	5.5–23.5	0.751–1.070	−0.90	7.3
3D	16×8	17.4–31.1	0.899–1.122	−0.83	26.7

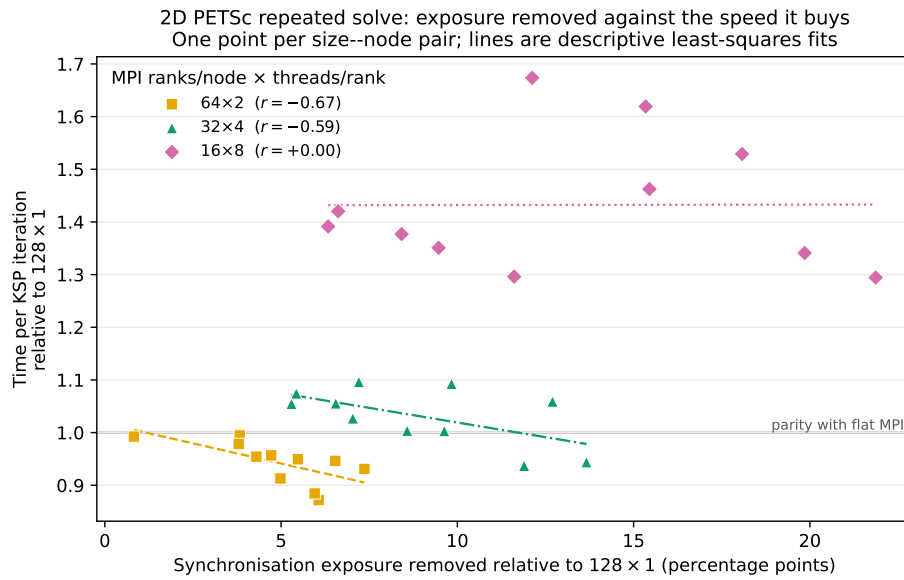


Figure 5.4: 2D exposure removed relative to flat MPI against the resulting time per iteration.

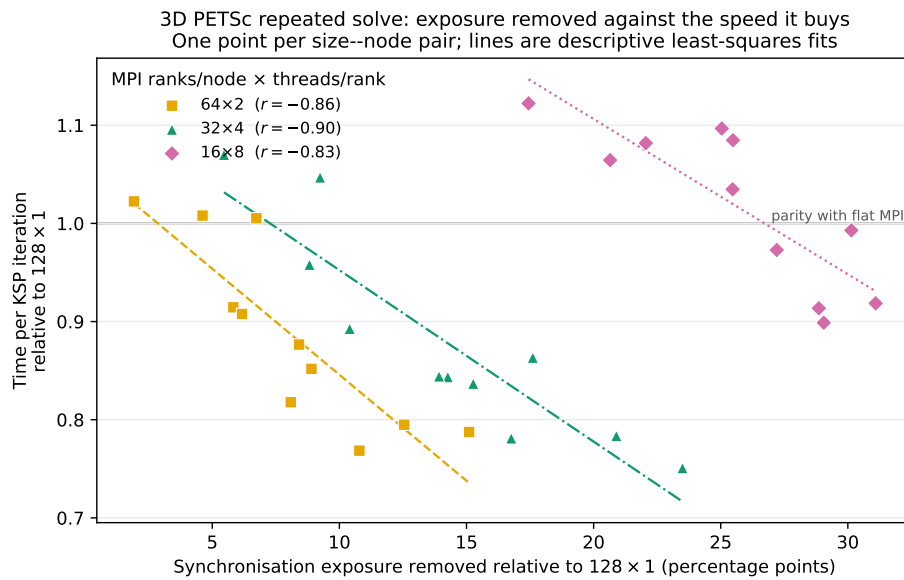


Figure 5.5: 3D exposure removed relative to flat MPI against the resulting time per iteration.

5.4.4 What This Does and Does Not Establish

Chapter 6

Discussion

Chapter 7

Conclusion and Future Work

Appendix A

Supplementary Figures

A.1 Weak-scaling throughput

Figure A.1 and Figure A.2 give raw repeated-solve throughput for the weak-scaling campaign of Section 4.2. Throughput rises almost linearly with the node count along both chains in both dimensions. The four layouts differ by only a few per cent against a range of three orders of magnitude, so these figures cannot carry the layout comparison. Figure 4.1 and Figure 4.3 do that instead.

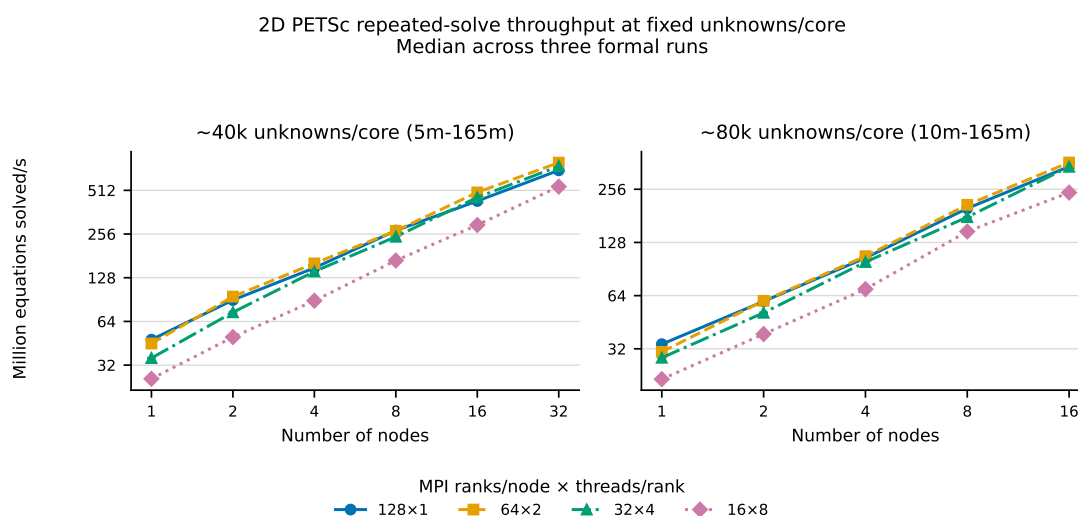


Figure A.1: Median 2D repeated-solve throughput along the two unknowns/core chains.

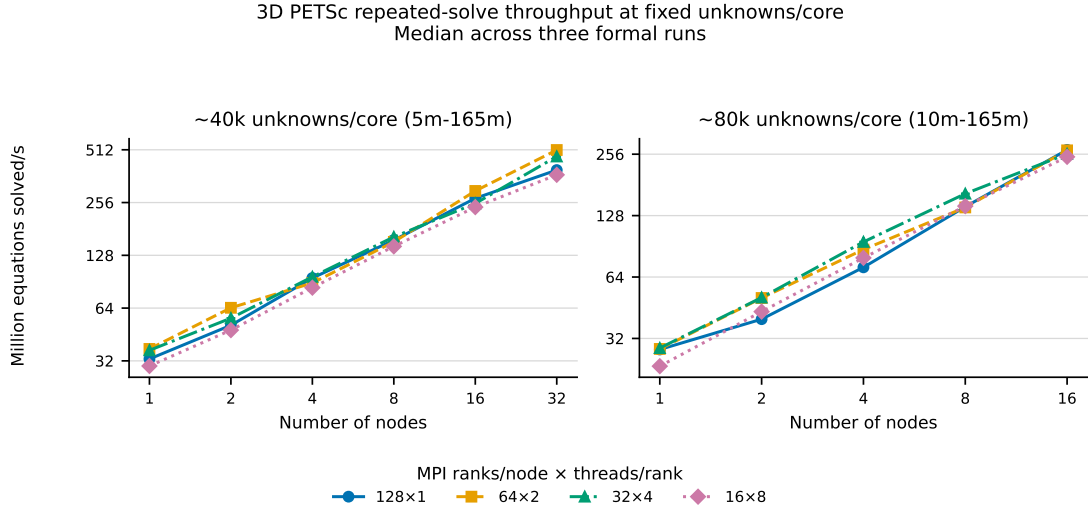


Figure A.2: Median 3D repeated-solve throughput along the two unknowns/core chains.

A.2 Fixed-size runtime grids

Section 4.1.2 presented these two campaigns as curves. Figure 4.5 showed the four 2D layouts falling together across the whole range and still falling at 32 nodes, and Figure 4.6 showed the 3D layouts separating instead, as three of the four reached a minimum and turned upward.

Table A.1 and Table A.2 give every point behind those two figures.

Table A.1: 2D fixed-size campaign, 4500^2 and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row.

Layout	Median s/solve at N nodes						16 $\rightarrow$ 32	Spread
	1	2	4	8	16	32		
128 $\times$ 1	0.7704	0.3399	0.1351	0.0558	0.0313	0.0282	1.11	7.9%
64 $\times$ 2	0.8260	0.3365	0.1265	0.0541	0.0348	0.0277	1.26	2.7%
32 $\times$ 4	0.8839	0.3944	0.1465	0.0632	0.0388	0.0317	1.22	10.8%
16 $\times$ 8	1.1325	0.5223	0.2272	0.0973	0.0568	0.0409	1.39	4.8%

Table A.2: 3D fixed-size campaign, 273^3 and 20.25M unknowns, median seconds per solve over three runs at every layout and node count. The final columns give the 16-to-32-node speedup and the largest observed three-repeat spread on that row.

Layout	Median s/solve at N nodes						16→32	Spread
	1	2	4	8	16	32		
128×1	0.9066	0.5149	0.2208	0.0997	0.1776	0.2599	0.68	4.2%
64×2	0.8288	0.4041	0.2342	0.0936	0.0748	0.1703	0.44	3.2%
32×4	0.8022	0.4028	0.2145	0.1192	0.0652	0.0747	0.87	11.1%
16×8	0.9707	0.4718	0.2475	0.1330	0.1017	0.0681	1.49	7.6%

Appendix B

Stuff which is too detailed

Appendices should contain all the material which is considered too detailed to be included in the main body but which is, nevertheless, important enough to be included in the thesis.

Appendix C

Stuff which no-one will read

Some people include in their thesis a lot of detail, particularly computer code, which no-one will ever read. You should be careful that anything like this you include should contain some element of uniqueness which justifies its inclusion.

Bibliography

- [1] ARCHER2 Service. *ARCHER2 Hardware Documentation*. 2022. URL: <https://docs.archer2.ac.uk/user-guide/hardware/> (visited on 04/05/2026).
- [2] ARCHER2 Service. *ARCHER2 Performance Tuning Guide*. 2022. URL: <https://docs.archer2.ac.uk/user-guide/tuning/> (visited on 08/03/2026).